processed).
#14 Add q as the next point in the ordering.
#15 Update the queue (by adding all points directly reachable from q) and the reachability distance for q's neighbors.
#18 Return the ordering for dataset's points and their reachability distances.

The algorithm's core (also illustrated in figure 12.16 on a simplified example) consists of a cycle through each point in the dataset; entries are processed in chunks of contiguous, reachable points (the starting point is chosen either randomly or according to its position in the dataset), and non-core points are "skipped" (similarly to what happened in DBSCAN).

The points that are reachable from the ones already processed are kept in a priority queue and extracted according to their reachability distance (the smallest reachability distance from any of the processed points is on top of the queue, as we have already seen).

There is a key piece of code still missing, the `updateQueue` method, that's where the reachability distances are actually updated (together with the priority queue): listing 12.8 bridges this gap. The main purpose of this method is going through all points q in the ε-neighborhood of a point p (the point that is currently being processed) and update q's reachability distances by checking if p is closer to it than any of the other points previously processed.